

Locating Web Elements

Md. Ehsanul Haque

Senior Software Quality Assurance Engineer

Vivasoft Ltd.

Agenda

- Introduction to Locators
- Types of Locators
- Best Practices for Creating Efficient Locators
- Hands-On Practices



Introduction to Locators

When automating web applications using Selenium, interacting with web elements (buttons, input fields, links, etc.) is essential. **Locators** help identify these elements on a webpage.

What are Selenium Locators?

Selenium locators are techniques used to find and interact with elements in a web application. They allow us to perform actions like clicking buttons, entering text, and verifying content.

Types of Locators

- **ID Locator:**

- Uses the `id` attribute of an HTML element.
- The fastest and most reliable locator, when `id` is unique.
- Example: `driver.find_element(BY.ID("username"));`

- **Name Locator:**

- Finds elements based on the `name` attribute.
- Works well when the `name` is unique.
- Example: `driver.find_element(BY.NAME("password"));`

- **Class Name Locator:**

- Selects elements using the `class` attribute/name.
- Useful for elements that have unique class names.
- Example: `driver.find_element(BY.CLASS_NAME("login-btn"));`

- **Tag Name Locator:**

- Finds elements BY their tag names.
- Commonly used to fetch multiple elements (e.g., all links on a page).
- Example: `driver.find_element(BY.TAG_NAME("input"));`

- **Link Text Locator:**

- Used to locate links (`<a>` elements) BY their text.
- Example: `driver.find_element(BY.LINK_TEXT("Forgot Password"));`

- **Partial Link Text Locator:**

- Similar to Link Text but works with partial matches.
- Useful when only part of the text is known.
- Example: `driver.find_element(BY.PARTIAL_LINK_TEXT("Forgot"));`

Types of Locators

- **XPath (XML Path Language):**

- Can locate elements using attributes, relationships, and text values.
- Absolute XPath:

```
driver.find_element(BY.xpath("/html/body/div/input"));
```
- Relative XPath:

```
driver.find_element(BY.xpath("//input[@id='username']"));
```

- **CSS Selector:**

- A powerful and efficient way to locate elements using CSS properties.
- Example:

```
driver.find_element(BY.cssSelector("input#username"));
```

Practical Selenium

Examples of Locators

Let's use different types of locators in Selenium automation.

- ID Locator
 - Name Locator
 - Class Name Locator
 - Tag Name Locator
 - Link Text Locator
 - Partial Link Text Locator
 - CSS Selector
-

XPath in Selenium

What is XPath?

XPath (XML Path Language) is used to navigate and locate elements in an XML or HTML document. In Selenium, XPath helps find elements when **id**, **name**, or other simple locators are not sufficient.

Types of XPath

XPath can be broadly categorized into

- **Absolute XPath**
- **Relative XPath.**

1. Absolute XPath

- It provides the full path from the root element (/html).
- It is very fragile – any change in the structure breaks it.
- **Example:** `driver.find_element (BY.xpath("/html/body/div[1]/div[2]/input")).sendKeys("Test");`
- **Downside:** If the page structure changes, this XPath may fail.

2. Relative XPath

- Starts from anywhere in the document (not from the root).
- More flexible and recommended over absolute XPath.
- **Example:** `driver.find_element (BY.xpath("//input[@id='username']")).sendKeys("TestUser");`

Basic XPath Syntax

`//tagname`

- Selects all elements with the given tag

- `//input` (Selects all `<input>` elements)

`//tagname[@attribute='value']`

- Finds elements BY attribute

- `//input[@type='password']`
- `//input[@id='email']`

`//tagname[text()='text_value']`

- Selects element BY text

- `//button[text()='Login']`

`//tagname[contains(@attribute, 'value')]`

- Finds elements that partially match an attribute

- `//input[contains(@placeholder, 'Email')]`

Basic XPath Syntax

`//tagname[starts-with(@attribute, 'value')]`

- Finds elements where attribute starts with given value

• `//input[starts-with(@id, 'user')]`

`//tagname[@attribute1='value1' and @attribute2='value2']`

- Selects elements with multiple conditions

• `//input[@type='text' and @name='username']`

XPath with Axes

XPath axes help navigate through the DOM structure.

parent::

- Selects the parent of the current node

- `//input[@id='password']/parent::div`

child::

- Selects direct child elements

- `//div[@class='form-group']/child::input`

following::

- Selects all elements after the current node

- `//input[@id='password']/preceding::label`

preceding::

- Selects all elements before the current node

- `//input[contains(@placeholder, 'Email')]`

XPath with Axes

ancestor::

- Selects all ancestors of an element

- `//input[@id='login']/ancestor::form`

descendant::

- Selects all descendants (children, grandchildren, etc.)

- `//div[@class='container']/descendant::input`

Practical Selenium

Examples of XPath

Let's use different XPath types in Selenium automation.

- Locate an Input Field BY ID
 - Locate an Element BY Text
 - Using `contains()` Function
 - Using `starts-with()` Function
 - Using Parent and Child Axes
 - Using following and preceding Axes
 - Multiple Conditions in XPath
-

Best Practices for Writing XPath in Selenium

1. **Prefer ID over other locators** – It is unique and fast.
2. **Use meaningful attributes** – Prefer stable attributes like name or class instead of dynamically generated ones.
3. **Avoid absolute XPath** – Use relative XPath for better maintainability.
4. **Use contains and starts-with in XPath** to handle dynamic elements.
 - a. Example:
`//button[contains(text(),'Login')]`
 - b. Example: `//input[starts-with(@id,'user_')]`
5. **Use CSS selectors for performance** – Faster than XPath in most cases.
 - a. Example: `input[name='email']` instead of `//input[@name='email']`
- ~~6. **Use parent-child relationships** – To make selectors more readable and reliable.~~

Thank you!